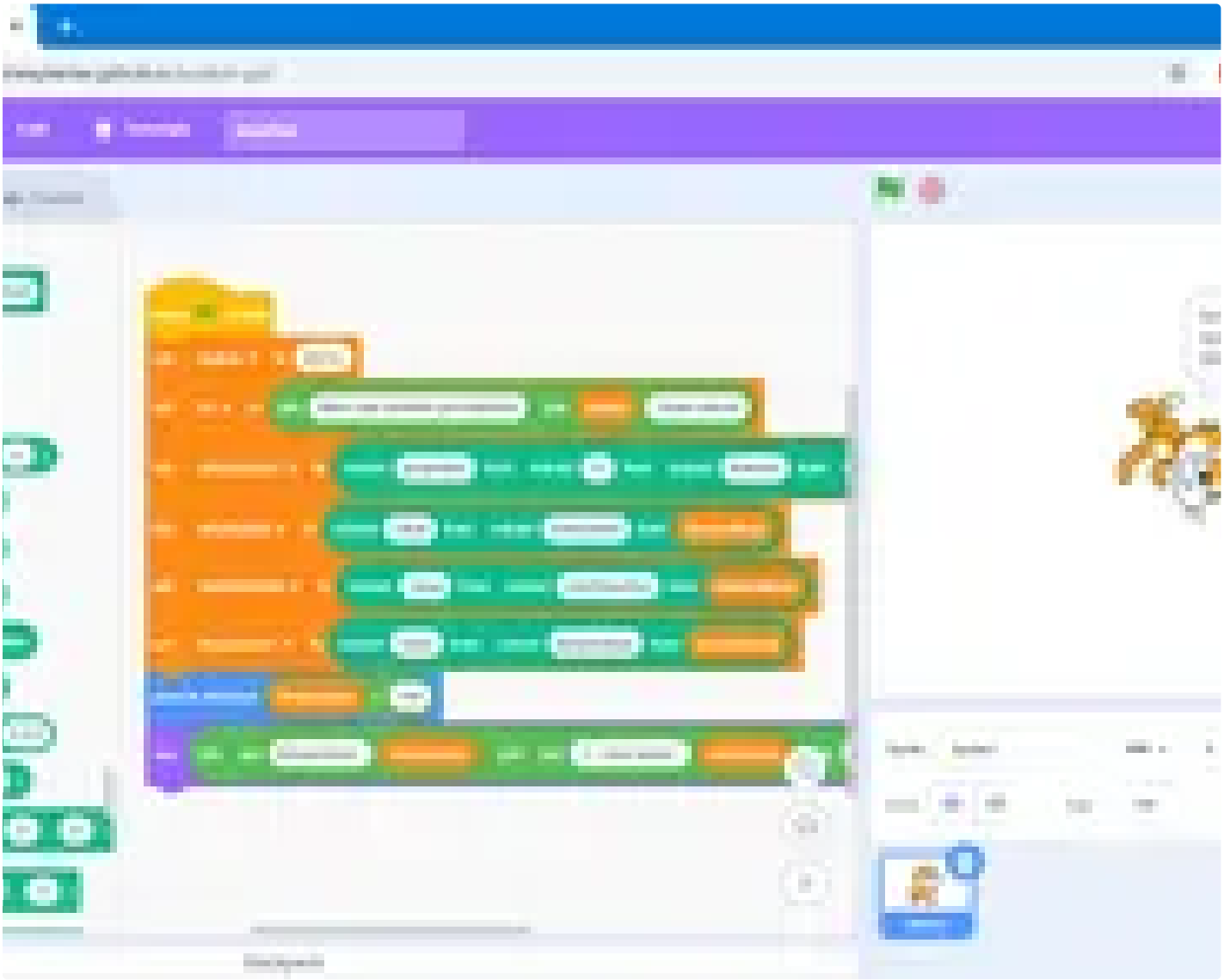
Scratch 3.0 Extensions

By arpruss in Circuits > Computers 30,143 22 4

(cc) BY



Scratch extensions are pieces of Javascript code that add new blocks to Scratch. While Scratch is bundled with a bunch of official extensions, there isn't an official mechanism for adding user-made extensions.

When I was making my Minecraft controlling extension for Scratch 3.0, I found it difficult to get started. This Instructable collects together information from various sources (especially this), plus a few things I discovered myself.

You need to know how to program in Javascript and how to host your Javascript on a website. For the latter, I recommend GitHub Pages.

The main trick is to use SheepTester's mod of Scratch which lets you load extensions and plugins.

This Instructable will guide you through making two extensions:

- **Fetch:** loading data from a URL and extracting JSON tags, for instance for loading weather data
- **SimpleGamepad:** using a game controller in Scratch (a more sophisticated version is here).

Step 1: Two Types of Extensions

There are two types of extensions which I will call "unsandboxed" and "sandboxed". Sandboxed extensions run as Web Workers, and as a result have significant limitations:

- Web Workers cannot access the globals in the **window** object (instead, they have a global **self** object, which is much more limited), so you cannot use them for things like gamepad access.
- Sandboxed extensions do not have access to the Scratch runtime object.
- Sandboxed extensions are much slower.
- Javascript console error messages for sandboxed extensions are more cryptic in Chrome.

On the other hand:

- Using other people's sandboxed extensions is safer.
- Sandboxed extensions are more likely to work with any eventual official extension loading support.
- Sandboxed extensions can be tested without uploading to a web server by encoding into a **data://** URL.

The official extensions (such as the Music, Pen, etc.) are all unsandboxed. The constructor for the extension gets the **runtime** object from Scratch, and **window** is fully accessible.

The Fetch extension is sandboxed, but the Gamepad one needs the **navigator** object from **window**.

Step 2: Writing a Sandboxed Extension: Part I

To make an extension, you create a class which encodes information about it, and then add a bit of code to register the extension.

The main thing in the extension class is a **getInfo()** method which returns an object with the required fields:

- **id**: the internal name of the extension, must be unique for each extension
- **name**: the friendly name of the extension, showing up in Scratch's list of blocks
- **blocks**: a list of objects describing the new custom block.

And there is an optional **menus** field which doesn't get used in Fetch but will be used in Gamepad.

So, here is the basic template for Fetch:

```
class ScratchFetch {
    constructor() {
    }

    getInfo() {
        return {
            "id": "Fetch",
            "name": "Fetch",
            "blocks": [ /* add later */ ]
        }
    }

    /* add methods for blocks */
}

Scratch.extensions.register(new ScratchFetch())
```

Step 3: Writing a Sandboxed Extension: Part II

Now, we need to create the list of blocks in **getInfo()**'s object. Each block needs at least these four fields:

- **opcode**: this is the name of the method that is called to do the block's work
- **blockType**: this is the block type; the most common ones for extensions are:
  - "command": does something but doesn't return a value
  - "reporter": returns a string or number
  - "Boolean": returns a boolean (note the capitalization)
  - "hat": event catching block; if your Scratch code uses this block, the Scratch runtime regularly polls the associated method which returns a boolean to say whether the event has happened
- **text**: this is a friendly description of the block, with the arguments in brackets, e.g., "fetch data from [url]"
- **arguments**: this is an object having a field for every argument (e.g., "url" in the above example); this object in turn has these fields:
  - **type**: either "string" or "number"
  - **defaultValue**: the default value to be pre-filled.

For instance, here is the **blocks** field in my Fetch extension:

```
"blocks": [
            {
                "opcode": "fetchURL",
                "blockType": "reporter",
                "text": "fetch data from [url]",
                "arguments": {
                    "url": {
                        "type": "string",
                        "defaultValue": "https://api.weather.gov/s
                    },
                }
            },
            {
                "opcode": "jsonExtract",
                "blockType": "reporter",
                "text": "extract [name] from [data]",
                "arguments": {
                    "name": {
                        "type": "string",
```

```
                                "defaultValue": "temperature"
                            },
                            "data": {
                                "type": "string",
                                "defaultValue": '{"temperature": 12.3}'
                            },
                        }
                    },
                ]
```

Here, we defined two blocks: **fetchURL** and **jsonExtract**. Both are reporters. The first pulls data from a URL and returns it, and the second extracts a field from JSON data.

Finally, you need to include the methods for two blocks. Each method takes an object as an argument, with the object including fields for all the arguments. You can decode these using curly braces in the arguments. For instance, here is one synchronous example:

```
jsonExtract({name,data}) {
    var parsed = JSON.parse(data)
    if (name in parsed) {
        var out = parsed[name]
        var t = typeof(out)
        if (t == "string" || t == "number")
            return out
        if (t == "boolean")
            return t ? 1 : 0
        return JSON.stringify(out)
    }
    else {
        return ""
    }
}
```

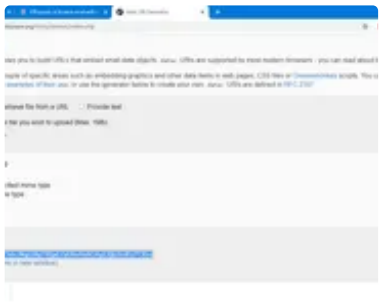
The code pulls the **name** field from the JSON **data**. If the field contains a string, number or boolean, we return that. Otherwise, we re-JSONify the field. And we return an empty string if the **name** is missing from the JSON.

Sometimes, however, you may want to make a block that uses an asynchronous API. The **fetchURL()** method uses the **fetch** API which is asynchronous. In such a case, you should return a promise from your method that does the work. For instance:

```
fetchURL({url}) {
    return fetch(url).then(response => response.text())
}
```

That's it. The full extension is here.

Step 4: Using a Sandboxed Extension



There are two ways of using sandboxed extension. First, you can upload it to a web server, and then load it into SheepTester's Scratch mod. Second, you can encode it into a data URL, and load that into the Scratch mod. I actually use the second method quite a bit for testing, as it avoids worries about older versions of the extension getting cached by the server. Note that while you can host javascript from Github Pages, you cannot do so directly from an ordinary github repository.

My fetch.js is hosted at **https://arpruss.github.io/fetch.js** . Or you can convert your extension to a data URL by uploading it here and then copy it to the clipboard. A data URL is a giant URL that holds a whole file in it.

Go to SheepTester's Scratch mod. Click on the Add Extension button in the lower-left corner. Then click on "Choose an extension", and enter your URL (you can paste in the whole giant data URL if you like).

If all went well, you will have an entry for your extension on the left-side of your Scratch screen. If things didn't go well, you should open your Javascript console (shift-ctrl-J in Chrome) and try to debug the issue.

Above you will find some example code that fetches and parses JSON data from the KNYC (in New York) station of the US National Weather Service, and displays it, while turning the sprite to face the same way that the wind is blowing. The way I made it was by fetching the data into a web browser, and then figuring out the tags. If you want to try a different weather station, enter a nearby zip code into the search box at **weather.gov**, and the weather page for your location should give you a four letter station code, which you can use in place of **KNYC** in the code.

You can also include your sandboxed extension right in the URL for SheepTester's mod by adding a "?url=" argument. For instance:

```
https://sheeptester.github.io/scratch-gui/?url=https://arpruss.github.io/fetch
```

Step 5: Writing an Unsandboxed Extension: Introduction

The constructor of an unsandboxed extension gets passed a Runtime object. You can ignore it or use it. One use of the Runtime object is to use its **currentMSecs** property to synchronize events ("hat blocks"). As far as I can tell, all the event block opcodes are polled regularly, and each round of the polling has a single **currentMSecs** value. If you need the Runtime object, you will probably start your extension with:

```
class EXTENSIONCLASS {
    constructor(runtime) {
        this.runtime = runtime
        ...
    }
    ...
}
```

All the standard **window** object things can be used in the unsandboxed extension. Finally, your unsandboxed extension should end with this bit of magic code:

```
(function() {
    var extensionInstance = new EXTENSIONCLASS(window.vm.extensionManager.runt
    var serviceName = window.vm.extensionManager._registerInternalExtension(ex
    window.vm.extensionManager._loadedExtensions.set(extensionInstance.getInfo
})()
```

where you should replace **EXTENSIONCLASS** with your extension's class.

Step 6: Writing an Unsandboxed Extension: Simple Gamepad

Let's now make a simple gamepad extension that provides a single event ("hat") block for when a button is pressed or released.

During each event block polling cycle, we will save a timestamp from the runtime object, and the previous and current gamepad states. The timestamp is used to recognize if we have a new polling cycle. So, we start with:

```
class ScratchSimpleGamepad {
    constructor(runtime) {
        this.runtime = runtime
        this.currentMSecs = -1
        this.previousButtons = []
        this.currentButtons = []
    }
    ...
}
```

We will have one event block, with two inputs--a button number and a menu to select whether we want the event to trigger on press or release. So, here is our method:

```
   getInfo() {
        return {
            "id": "SimpleGamepad",
            "name": "SimpleGamepad",
            "blocks": [{
                        "opcode": "buttonPressedReleased",
                        "blockType": "hat",
                        "text": "button [b] [eventType]",
                        "arguments": {
                            "b": {
                                "type": "number",
                                "defaultValue": "0"
                            },
                            "eventType": {
                                "type": "number",
                                "defaultValue": "1",
                                "menu": "pressReleaseMenu"
                            },
                        },
                    },
            ],
            "menus": {
```

```
                "pressReleaseMenu": [{text:"press",value:1}, {text:"release",v
            }
        };
    }
```

I think the values in the drop-down menu still get passed to the opcode function as strings, despite being declared as numbers. So explicitly compare them against the values specified in the menu as needed.

We now write a method that updates the button states whenever a new event polling cycle happens:

```
    update() {
        if (this.runtime.currentMSecs == this.currentMSecs)
            return // not a new polling cycle
        this.currentMSecs = this.runtime.currentMSecs
        var gamepads = navigator.getGamepads()
        if (gamepads == null || gamepads.length == 0 || gamepads[0] == null) {
            this.previousButtons = []
            this.currentButtons = []
            return
        }
        var gamepad = gamepads[0]
        if (gamepad.buttons.length != this.previousButtons.length) {
            // different number of buttons, so new gamepad
            this.previousButtons = []
            for (var i = 0; i < gamepad.buttons.length; i++)
                this.previousButtons.push(false)
        }
        else {
            this.previousButtons = this.currentButtons
        }
        this.currentButtons = []
        for (var i = 0; i < gamepad.buttons.length; i++)
            this.currentButtons.push(gamepad.buttons[i].pressed)
    }
```

Finally, we can implement our event block, by calling the **update()** method and then checking if the requisite button has just been pressed or released, by comparing current and previous button states:

```
    buttonPressedReleased({b,eventType}) {
        this.update()
        if (b < this.currentButtons.length) {
            if (eventType == 1) { // note: this will be a string, so better to
                if (this.currentButtons[b] && ! this.previousButtons[b]) {
```

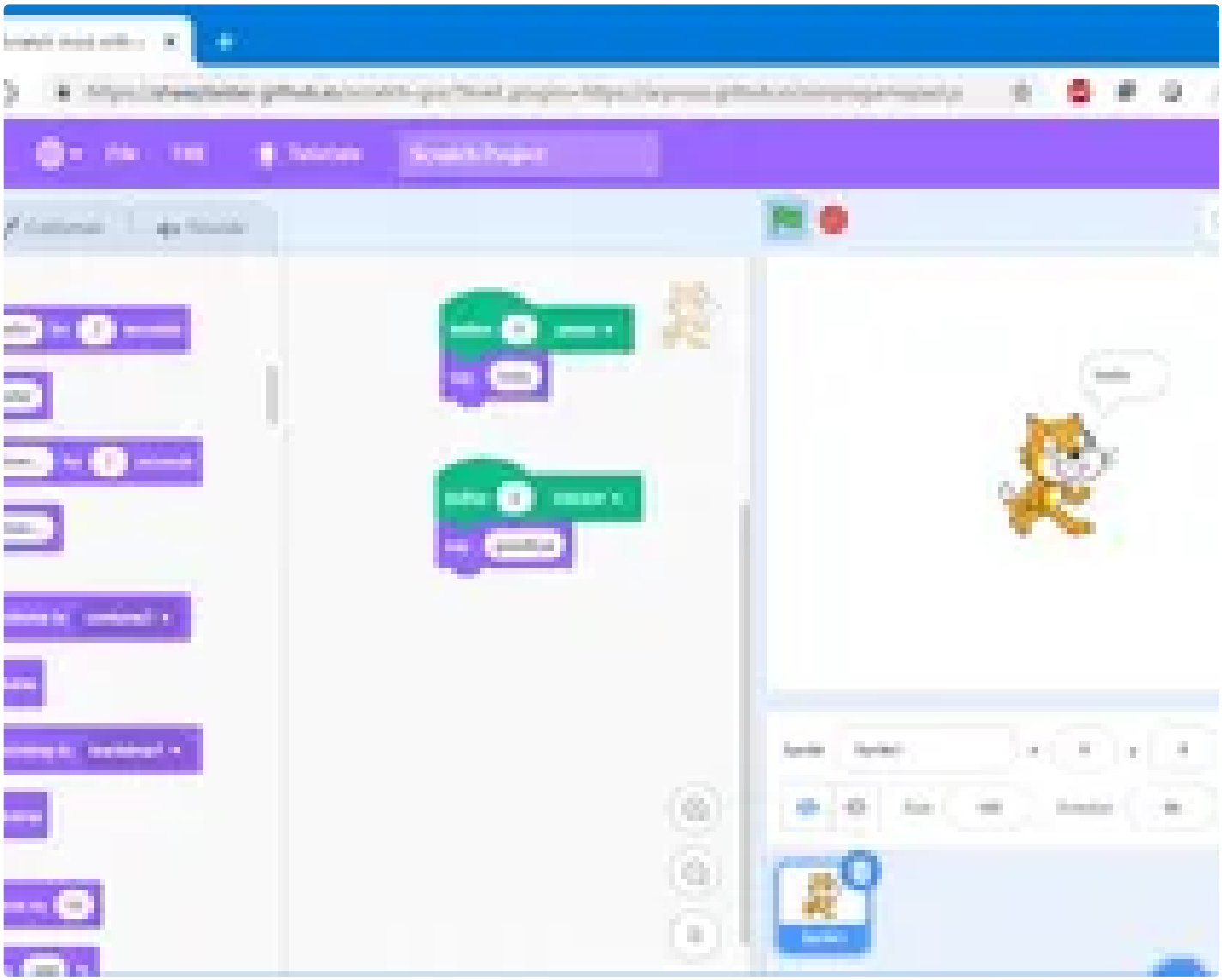
```
                return true
            }
        }
        else {
            if (!this.currentButtons[b] && this.previousButtons[b]) {
                return true
            }
         }
    }
    return false
}
```

And finally we add our magic extension registration code after defining the class:

```
(function() {
    var extensionInstance = new ScratchSimpleGamepad(window.vm.extensionManage
    var serviceName = window.vm.extensionManager._registerInternalExtension(ex
    window.vm.extensionManager._loadedExtensions.set(extensionInstance.getInfo
})()
```

You can get the full code here.

Step 7: Using an Unsandboxed Extension



Once again, host your extension somewhere, and this time load it with the **load_plugin=** rather than **url=** argument to SheepTester's Scratch mod. For instance, for my simple Gamepad mod, go to:

`https://sheeptester.github.io/scratch-gui/?load_plugin=https://arpruss.github.`

(By the way, if you want a more sophisticated gamepad, just remove "simple" from the above URL, and you will have rumble and analog axis support.)

Again, the extension should appear on the left side of your Scratch editor. Above is a very simple Scratch program that says "hello" when you press button 0 and "goodbye" when you release it.

Step 8: Dual-compatibility and Speed

I have noticed that extension blocks run an order of magnitude faster using the loading method I used for unsandboxed extensions. So unless you care about the security benefits of running in a Web Worker sandbox, your code will benefit from being loaded with the **?load_plugin=URL** argument to SheepTester's mod.

You can make a sandboxed extension compatible with both loading methods by using the following code after defining the extension class (change **CLASSNAME** to the name of your extension class):

```
(function() {
    var extensionClass = CLASSNAME
    if (typeof window === "undefined" || !window.vm) {
        Scratch.extensions.register(new extensionClass())
    }
    else {
        var extensionInstance = new extensionClass(window.vm.extensionManager.
        var serviceName = window.vm.extensionManager._registerInternalExtensio
        window.vm.extensionManager._loadedExtensions.set(extensionInstance.get
    }
})()
```